Project - High Level Design
on
Real Estate Support Triage Agent

Course Name: Agentic AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	HARSHIT MITTAL	EN22CS301401
2.	HARSHAL DESHMUKH	EN22CS301396
3.	HARSHIT CHOUDHARY	EN22CS301400
4.	SALONEE CHOUHAN	EN23CS3L1017
5.	HARSHIT SHARMA	EN22CS301403
6.	HARSHIT BHARDWAJ	EN22CS301399

Group Name: Group 11D4

Project Number: AAI-47

Industry Mentor Name: Mr. Ashwin Krishnan CK

University Mentor Name: Prof. Pradeep Baniya

Academic Year: 2025-26

Table of Contents

1. Introduction.
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.

2. System Design.
   2.1. Application Design
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Components Design
   2.5. Key Design Considerations
   2.6. API Catalogue.

3. Data Design.
   3.1. Data Model
   3.2. Data Access Mechanism
   3.3. Data Retention Policies
   3.4. Data Migration

4. Interfaces
   4.1. User Interface
   4.2. Admin Interface
   4.3. API Interface

5. State and Session Management

6. Caching

7. Non-Functional Requirements
   7.1. Security Aspects
   7.2. Performance Aspects

8. References

1. Introduction

The AI-Powered Real Estate Customer Support Triage System is designed to automate the handling and classification of customer inquiries in a real estate environment. Real estate companies often receive a large number of customer queries related to buying, renting, property viewing, complaints, and general inquiries. Manually processing these queries can lead to delays and inefficiencies.

This system uses an Artificial Intelligence-based triage mechanism powered by a multi-agent architecture using CrewAI and the Gemini Large Language Model. The system automatically classifies inquiries, extracts relevant information such as property IDs and appointment dates, and generates professional draft responses.

The platform also includes user authentication, inquiry history tracking, and an admin dashboard to manage and monitor customer inquiries effectively.

1.1 Scope of the Document

This document provides the High-Level Design (HLD) of the Real Estate AI Triage System. It describes the overall system architecture, key components, interaction between modules, data flow, system interfaces, and major design considerations.
The purpose of this document is to provide a clear understanding of how the system is structured and how different modules interact with each other.

1.2. Intended Audience

This document is intended for:

- Software developers implementing the system
- Project evaluators
- Academic mentors
- System architects
- Stakeholders interested in understanding the system architecture

1.3. System Overview

The Real Estate AI Triage System is a full-stack web application that enables users to submit property-related inquiries and receive automated responses generated by an AI triage pipeline.

The system architecture consists of the following major components:

i. Frontend Interface
A React-based web interface that allows users to register, log in, submit inquiries, and view their previous requests.

ii. Backend Server
A FastAPI-based backend responsible for handling API requests, authentication, business logic, and communication with the AI pipeline.

iii. AI Processing Layer
A CrewAI multi-agent pipeline powered by the Gemini language model that processes user inquiries.

iv. Database
A PostgreSQL database used to store user accounts and triage records.

v. Admin Dashboard
An administrative interface that allows administrators to monitor and manage incoming customer inquiries.

---

2. System Design

The system follows a layered architecture approach consisting of four primary layers:

i. Presentation Layer
Implemented using React (Vite). This layer handles user interaction and displays results.

ii. Application Layer
Implemented using FastAPI. This layer processes incoming requests, performs authentication, and interacts with the AI processing pipeline.

iii. AI Processing Layer
Implemented using CrewAI agents integrated with the Gemini API to analyze inquiries.

iv. Data Layer
Implemented using PostgreSQL with Alembic migrations for database version control.
The layered architecture ensures separation of concerns, scalability, and maintainability.

2.1 Application Design

The Real Estate AI Triage System follows a modular application architecture where the system is divided into independent functional modules. Each module is responsible for a specific set of operations such as authentication, inquiry processing, AI triage, and administrative management. This modular design ensures scalability, maintainability, and separation of concerns.

The application consists of four primary logical modules:
- **Authentication Module**
- **User Inquiry Management Module**
- **AI Triage Processing Module**
- **Administrative Management Module**

Each module interacts with the backend services implemented using FastAPI and communicates with the PostgreSQL database for data persistence.

i. Authentication Module

The authentication module manages user identity verification and access control. New users are required to register before accessing the system.
During registration, the following details are collected:

- Full name
- Phone number
- Password

Passwords are securely stored in the database in hashed format. The system uses **JWT (JSON Web Token)** based authentication for session management.
When a user logs in, the backend verifies the credentials and generates a JWT token. This token is returned to the frontend and stored on the client side. For every protected API request, the frontend includes this token in the authorization header, allowing the backend to validate the user's identity before granting access to protected resources.
The authentication module ensures that only authorized users can submit inquiries or access administrative functionality.

ii. User Inquiry Management Module

The User Inquiry Management Module handles all operations related to submitting and managing real estate inquiries.
After successful login, users can access the inquiry dashboard where they can:

- Submit a new inquiry related to real estate properties
- View previously submitted inquiries
- Track the status of each inquiry

When a user submits an inquiry, the message is sent to the backend through the triage API endpoint. The backend forwards this request to the AI processing pipeline for analysis.
Each inquiry is stored in the **triage_records** table in the database along with the AI-generated results. Users can view their past requests in the "My Requests" section, where the system displays details such as:

- Inquiry text
- Property ID
- Date of submission
- Current status (Unsolved, Action Taken, or Solved)

This module ensures that users can track the lifecycle of their inquiries within the platform.

iii. AI Triage Processing Module

The AI triage processing module is responsible for analyzing user inquiries using a multi-agent artificial intelligence pipeline.
This module is implemented using **CrewAI**, which orchestrates multiple specialized agents powered by the **Gemini language model**.
The AI pipeline consists of three agents:

1. **Classification Agent**
   Determines the urgency level and intent of the inquiry.
2. **Extraction Agent**
   Extracts structured entities from the inquiry such as property ID and appointment dates.
3. **Response Agent**
   Generates a professional draft response based on the classification and extracted information.

The pipeline processes the inquiry sequentially, producing a structured output that includes urgency level, intent, property reference, and a draft response. This structured result is then returned to the backend and stored in the database as a triage record.
This architecture allows the system to handle large volumes of customer inquiries while maintaining consistent and structured outputs.

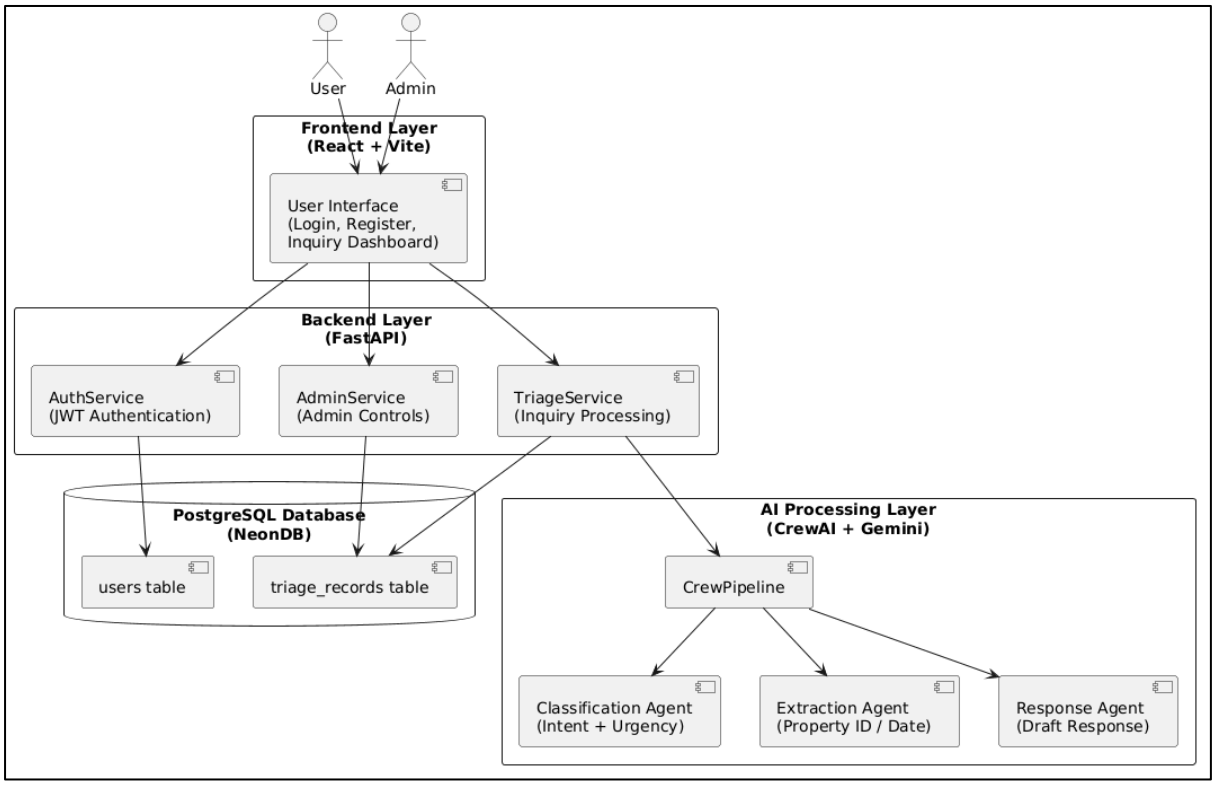
iv. Administrative Management Module
The Administrative Management Module allows system administrators to monitor and manage customer inquiries.
Administrators can access a dedicated dashboard that displays all incoming inquiries sorted by urgency level. The dashboard provides detailed information for each inquiry, including:

- User name
- Phone number
- Inquiry text
- Intent classification
- Property ID
- Generated response
- Current status

Administrators can update the status of each inquiry to reflect the progress of customer support activities. The available status options include:

- Unsolved
- Action Taken
- Solved

This module enables real estate companies to efficiently track customer interactions and prioritize urgent requests.



Architecture Diagram

2.2 Process Flow

The process flow describes how a user interaction travels through the system from the user interface to the backend services, AI processing pipeline, and database before returning the final response.

The system supports two primary workflows:

1. **User Inquiry Processing Workflow**
2. **Administrative Query Management Workflow**

Both workflows operate through the FastAPI backend which coordinates authentication, AI processing, and database operations.

i. User Inquiry Processing Workflow

The user inquiry workflow begins when a registered user logs into the system and submits a property-related inquiry.

1. The user accesses the web application through the React-based frontend.
2. The user logs in using their phone number and password.
3. The frontend sends the login request to the backend authentication service.
4. The backend verifies the credentials and generates a JWT token.
5. The JWT token is returned to the frontend and stored on the client side.
6. The user submits a real estate inquiry through the dashboard.
7. The frontend sends the inquiry request to the backend TriageService API endpoint with the JWT token attached in the authorization header.
8. The backend validates the token and forwards the inquiry to the AI processing pipeline.
9. The CrewPipeline orchestrates three agents:
    - Classification Agent determines urgency and intent.
    - Extraction Agent extracts property IDs or date references.
    - Response Agent generates a professional draft response.
10. The pipeline returns structured triage data to the backend.
11. The backend stores the inquiry and its processed results in the triage_records database table.
12. The generated response is returned to the frontend and displayed to the user.
13. The user can later view the inquiry and its status through the **My Requests** section of the dashboard.

ii. Administrative Query Management Workflow

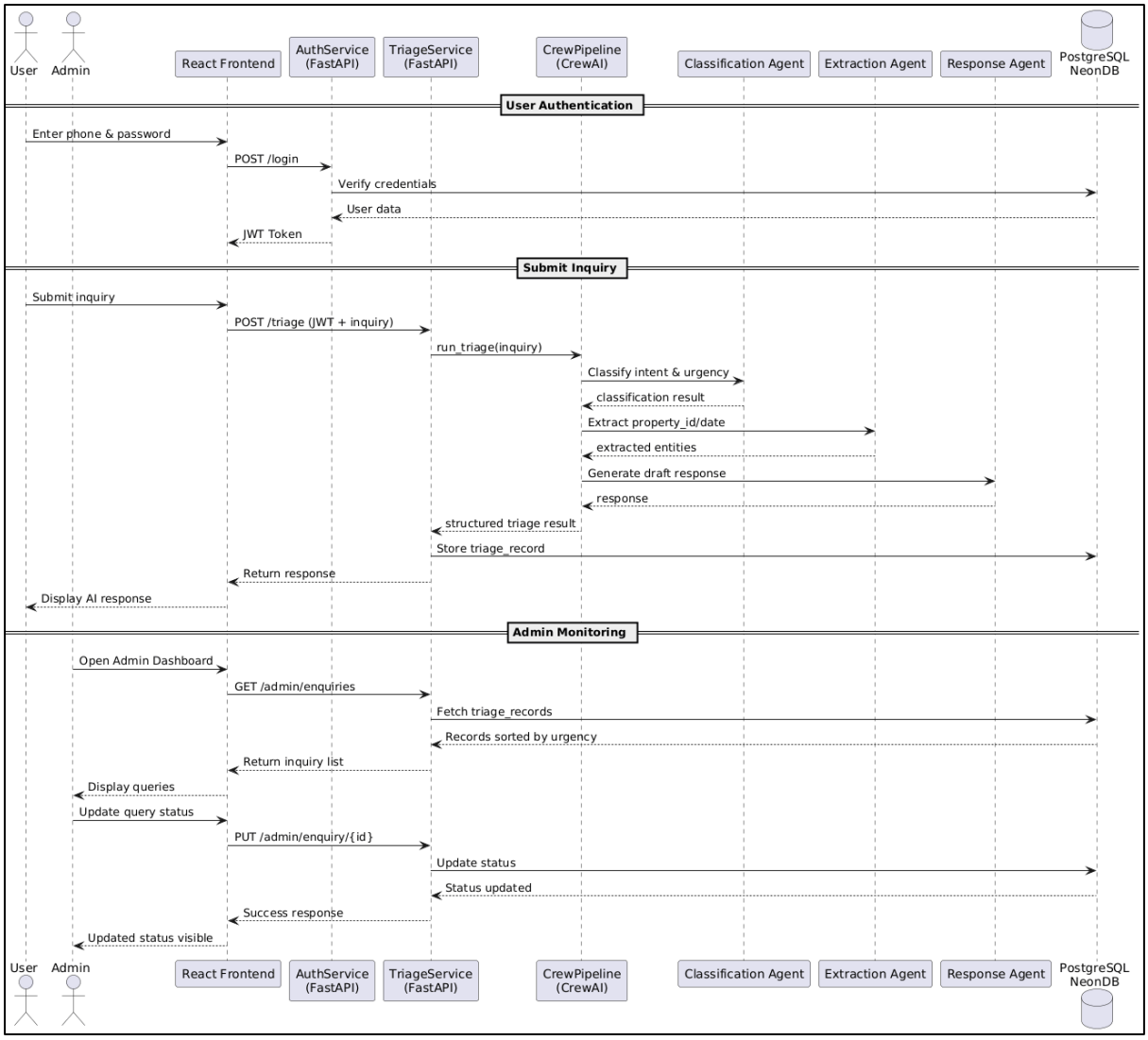
The administrative workflow allows administrators to monitor and manage all customer inquiries.

1. An administrator logs into the system using administrative credentials.
2. The backend authenticates the admin and verifies the is_admin flag associated with the user account.
3. The admin dashboard retrieves all inquiry records from the triage_records table.
4. The records are displayed in the interface sorted by urgency level.
5. Each record displays the following information:
    - User name
    - Phone number

- Inquiry message
- Intent classification
- Property reference
- AI-generated response
- Current status

6. The administrator can update the inquiry status to:
   - Unsolved
   - Action Taken
   - Solved
7. The backend updates the status field in the database and reflects the change in the dashboard.

This workflow ensures that real estate staff can efficiently track and resolve customer inquiries.



Process Flow Diagram

2.3 Information Flow

The information flow describes how data moves across different layers of the Real Estate AI Triage System. The system follows a layered architecture in which user inputs are processed by the frontend interface, forwarded to backend services, analyzed by the AI pipeline, and finally stored in the database.
The flow of information ensures that all system components communicate in a structured and secure manner.

i. User Interaction Flow

The information flow begins when a user interacts with the frontend application.

1. The user submits credentials or an inquiry through the React-based user interface.
2. The frontend sends the request to the FastAPI backend via REST API endpoints.
3. For authentication requests, the data is forwarded to the AuthService which validates user credentials against the database.
4. If authentication is successful, the backend generates a JWT token and returns it to the frontend.
5. The frontend stores this token and includes it in the authorization header for future requests.

This mechanism ensures secure communication between the frontend and backend.

ii. Inquiry Processing Flow

When a user submits a real estate inquiry, the following information flow occurs:

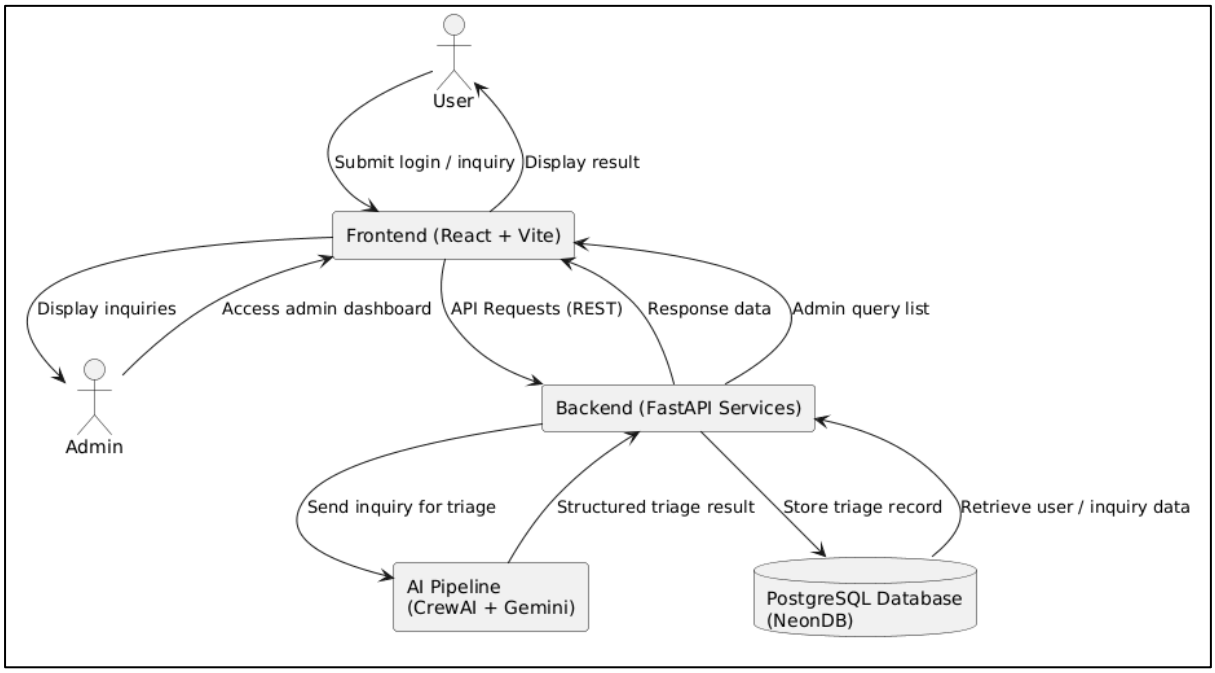
1. The inquiry message is sent from the frontend to the TriageService API endpoint.
2. The backend validates the JWT token to ensure the request is authenticated.
3. The inquiry text is passed to the AI processing layer through the CrewPipeline.
4. The pipeline orchestrates multiple agents that analyze the inquiry.
5. The agents produce structured output containing:
    - urgency level
    - intent classification
    - extracted property reference
    - generated response
6. The backend receives this structured result and stores it in the triage_records database table.
7. The backend returns the generated response to the frontend.
8. The frontend displays the result to the user.

iii. Administrative Data Flow

The admin dashboard retrieves information from the backend in order to monitor customer inquiries.

1. The administrator accesses the dashboard interface.
2. The frontend sends a request to retrieve all triage records.
3. The backend queries the triage_records table in the PostgreSQL database.
4. The retrieved records are sorted by urgency level.
5. The backend sends the processed data to the frontend where it is displayed in the admin dashboard.
6. When an admin updates the status of a query, the updated status is sent back to the backend and stored in the database.

This ensures that administrative users can efficiently manage and track customer inquiries.



Data Flow Diagram

2.4 Components Design

This section describes the major components of the system and their primary responsibilities. Each component represents a logical unit responsible for a specific functionality within the architecture.

The system is organized into the following key components:

- Frontend Application
- Authentication Service
- Triage Processing Service
- AI Processing Pipeline
- Administrative Service
- Database Layer

These components work together to process user inquiries, generate AI-driven responses, and manage inquiry records.

i. Frontend Application

The frontend application is implemented using **React with Vite** and serves as the user interaction layer of the system.
It provides the following interfaces:

- User registration interface
- User login interface
- Inquiry submission dashboard
- Inquiry history view
- Administrative dashboard for managing queries

The frontend communicates with backend services through REST APIs and includes JWT tokens in request headers for authentication.

ii. Authentication Service

The authentication service is responsible for managing user identity and access control.
Its responsibilities include:

- User registration
- Credential verification during login
- Password hashing and verification
- JWT token generation
- Authorization validation for protected API endpoints

The service interacts with the **users table** in the database to store and retrieve account information.

iii. Triage Processing Service

The Triage Processing Service acts as the central backend service responsible for handling inquiry submissions.
Its responsibilities include:

- Receiving inquiry requests from the frontend
- Validating authentication tokens
- Forwarding inquiries to the AI processing pipeline
- Receiving structured results from the pipeline
- Storing triage records in the database
- Returning generated responses to the frontend

This component serves as the bridge between the application layer and the AI processing layer.

iv. AI Processing Pipeline

The AI processing pipeline analyzes user inquiries and produces structured triage results.
The pipeline is implemented using **CrewAI**, which coordinates multiple specialized agents powered by the Gemini language model.
The pipeline consists of three agents:

Classification Agent: Determines the urgency level and intent of the inquiry.

Extraction Agent: Extracts structured entities such as property IDs or appointment dates from the inquiry text.

Response Agent: Generates a professional draft response that can be presented to the user.
The pipeline returns structured data that is later stored as a triage record.

v. Administrative Service

The administrative service manages system operations related to monitoring and updating inquiry records.
Its responsibilities include:

- Retrieving inquiry records for the admin dashboard
- Sorting inquiries based on urgency level
- Updating the status of customer inquiries
- Providing administrative access control

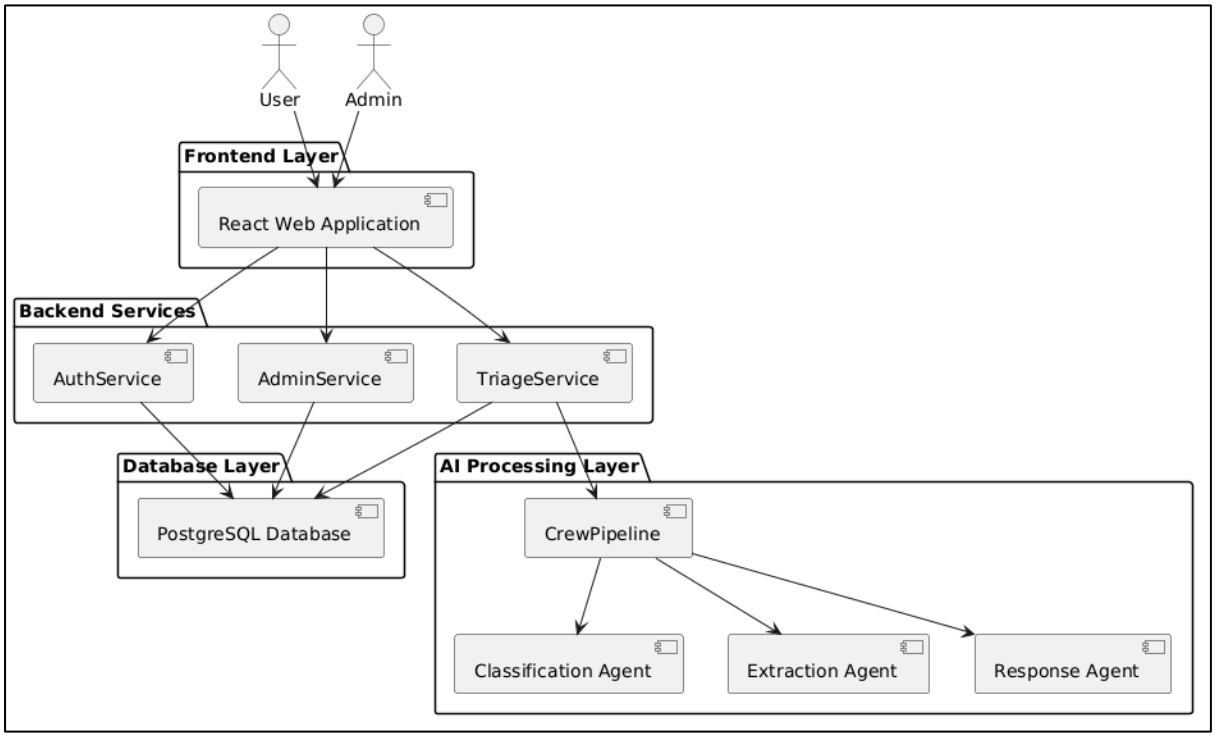
This service enables real estate staff to track customer interactions and prioritize critical requests.

vi. Database Layer
The database layer is implemented using **PostgreSQL hosted on NeonDB**.
It stores persistent application data in two primary tables:

Users Table: Stores registered user accounts and authentication details.

Triage Records Table: Stores user inquiries along with AI-generated analysis and administrative status.

Database schema migrations are managed using **Alembic**, which ensures consistent schema updates across environments.



Component Diagram

2.5 Key Design Considerations

The design of the Real Estate AI Triage System was guided by several architectural and operational considerations to ensure that the system remains scalable, secure, maintainable, and efficient. The following design principles influenced the system architecture and component organization.

i. Modular Architecture
The system follows a modular design where major functionalities are separated into independent services such as authentication, inquiry processing, AI triage, and administrative management.
This separation of responsibilities ensures that individual components can be developed, tested, and maintained independently. It also simplifies future enhancements, such as integrating additional AI agents or extending administrative capabilities.

ii. Layered System Design

The system architecture is divided into logical layers:

- Presentation Layer (React frontend)
- Application Layer (FastAPI backend services)
- AI Processing Layer (CrewAI agents with Gemini)
- Data Layer (PostgreSQL database)

This layered design improves system organization and prevents tight coupling between components. Each layer interacts only with the layers directly adjacent to it.

iii. AI-Driven Automation

A key design goal of the system is to automate the initial triage of customer inquiries using artificial intelligence.

The multi-agent AI pipeline enables the system to:

- classify inquiry urgency
- identify customer intent
- extract structured entities from natural language input
- generate draft responses automatically

This significantly reduces manual workload for real estate support teams while maintaining consistent processing of customer requests.

iv. Secure Authentication Mechanism

The system uses **JWT-based authentication** to secure API endpoints and protect user data. JWT tokens allow stateless authentication where the backend verifies user identity without maintaining session data on the server. This approach improves scalability and simplifies API security.

Additionally, user passwords are stored in hashed format to prevent exposure of sensitive credentials.

v. Scalability and Extensibility

The architecture allows future extensions without major structural changes. Examples of potential enhancements include:

- integrating additional AI agents for more advanced triage analysis
- adding notification systems for high urgency inquiries
- connecting the platform to external CRM systems used by real estate companies
- implementing analytics dashboards for inquiry trends

The modular service design makes these upgrades feasible without affecting existing components.

vi. Database Design Simplicity

The database schema was intentionally kept simple to support efficient query operations and reduce system complexity.

The system currently uses two primary tables:

- users
- triage_records

This structure supports the main operational requirements while maintaining clear relationships between users and their inquiries.

vii. Administrative Monitoring
The system includes a dedicated administrative interface that allows administrators to monitor incoming inquiries and update their status.
Sorting inquiries by urgency ensures that critical customer requests are prioritized. This design supports real-world operational workflows used by customer support teams.

2.6 API Catalogue

The backend of the Real Estate AI Triage System exposes several RESTful API endpoints implemented using FastAPI. These APIs enable user authentication, inquiry processing, administrative monitoring, and data retrieval.
All protected endpoints require a **JWT authentication token** issued during the login process. The token must be included in the Authorization header using the Bearer scheme.

i. User Registration API

Endpoint
POST /register

Description
This endpoint registers a new user in the system. The user provides their name, phone number, and password. The backend validates the phone number and stores the password in hashed form before saving the user record in the database.

Request Body
```
{
 "name": "John Doe",
 "phone_number": "9876543210",
 "password": "password123",
 "is_admin": false
}
```

Processing Steps
1. Validate the phone number format.
2. Check whether the phone number already exists.
3. Hash the password using a secure hashing function.
4. Store the user record in the database.

Response
```
{
 "message": "User registered successfully"
}
```

ii. User Login API

Endpoint
POST /login

Description
This endpoint authenticates a registered user. If the credentials are valid, the system generates

a JWT token that the frontend will use for subsequent authenticated requests.

Request Format
The login API uses OAuth2 password form authentication.
Example request parameters:
username = phone_number
password = user password

Processing Steps
1. Retrieve the user record using the provided phone number.
2. Verify the password using secure password verification.
3. Generate a JWT access token containing user identity and admin status.

Response
```
{
 "access_token": "jwt_token_here",
 "token_type": "bearer",
 "is_admin": false
}
```

iii. Inquiry Triage API

Endpoint
POST /triage

Description
This endpoint processes a user inquiry using the AI triage pipeline. It analyzes the inquiry text, classifies urgency and intent, extracts relevant entities, and generates a draft response. The resulting triage data is stored in the triage_records database table.

Authentication Required
Yes — JWT token required.

Request Body
```
{
 "message": "I want to schedule a visit for REF-1234 this weekend"
}
```

Processing Steps
1. Validate the JWT token.
2. Send the inquiry message to the AI triage pipeline.
3. Receive structured output from the AI system.
4. Store the triage record in the database.
5. Return the AI-generated result to the user.

Response Example
```
{
 "urgency": "Medium",
 "intent": "Viewing",
```

"property_id": "REF-1234",
"appointment_date": "Weekend",
"draft_response": "Dear Client, thank you for your inquiry..."
}

iv. Retrieve User Records API

Endpoint
GET /user/records

Description
This endpoint retrieves all inquiries submitted by the currently logged-in user. It allows users to track the history and status of their previous requests.

Authentication Required
Yes — JWT token required.

Response Example
[
{
"id": 5,
"urgency": "High",
"intent": "Complaint",
"inquiry": "The maintenance request for REF-1002 is still unresolved",
"property_id": "REF-1002",
"status": "Unsolved",
"created_at": "2025-02-12 10:45"
}
]

v. Retrieve Admin Records API

Endpoint
GET /admin/records

Description
This endpoint allows administrators to retrieve a list of inquiry records from the database. The records are sorted by urgency and creation time to help prioritize critical issues.

Authentication Required
Yes — Admin privileges required.

Response Example
[
{
"id": 12,
"name": "Rahul Sharma",
"phone_number": "9876543210",
"urgency": "High",

"intent": "Complaint",
"inquiry": "Water leakage in apartment REF-2056",
"property_id": "REF-2056",
"status": "Unsolved"
}
]

vi. Update Inquiry Status

Endpoint
PATCH /admin/records/{record_id}/status

Description
This endpoint allows an administrator to update the status of a triage record. The status indicates whether the issue has been resolved or is still pending.

Authentication Required
Yes — Admin privileges required.

Request Body
{
"status": "Solved"
}

Allowed Status Values
Unsolved
Action Taken
Solved

Response
{
"message": "Status updated successfully",
"status": "Solved"
}

---

3. Data Design

The Real Estate AI Triage System stores application data in a relational database implemented using **PostgreSQL hosted on NeonDB**. The database is responsible for storing user accounts, inquiry records, AI-generated analysis results, and administrative status updates.
The data layer is designed to maintain simplicity while supporting the core functionality of the system. The database schema ensures a clear relationship between users and their submitted inquiries while enabling efficient retrieval of records for both users and administrators.
The schema evolution and migration management are handled using **Alembic**, which ensures controlled database updates during development and deployment.

3.1 Data Model

The system currently uses two primary entities:
• Users
• Triage Records
These entities are related through a **one-to-many relationship**, where a single user can create multiple inquiry records.

i. Users Table
The users table stores authentication and identity information for all registered users in the system.
Key attributes include:

- **id** – Unique identifier for each user.
- **name** – Name of the user.
- **phone_number** – Unique phone number used for login.
- **hashed_password** – Securely stored hashed password.
- **is_admin** – Boolean flag indicating administrative privileges.

The phone number field is enforced as a unique constraint to prevent duplicate registrations.

ii. Triage Records Table
The triage_records table stores the processed results of each inquiry submitted by users.
Key attributes include:

- **id** – Unique identifier for each triage record.
- **user_id** – Foreign key referencing the users table.
- **inquiry** – Original inquiry message submitted by the user.
- **urgency** – AI-determined urgency level.
- **intent** – Classified intent of the inquiry.
- **property_id** – Extracted property reference from the inquiry.
- **appointment_date** – Extracted date reference if available.
- **draft_response** – AI-generated response suggestion.
- **status** – Administrative status of the inquiry.
- **created_at** – Timestamp indicating when the inquiry was created.

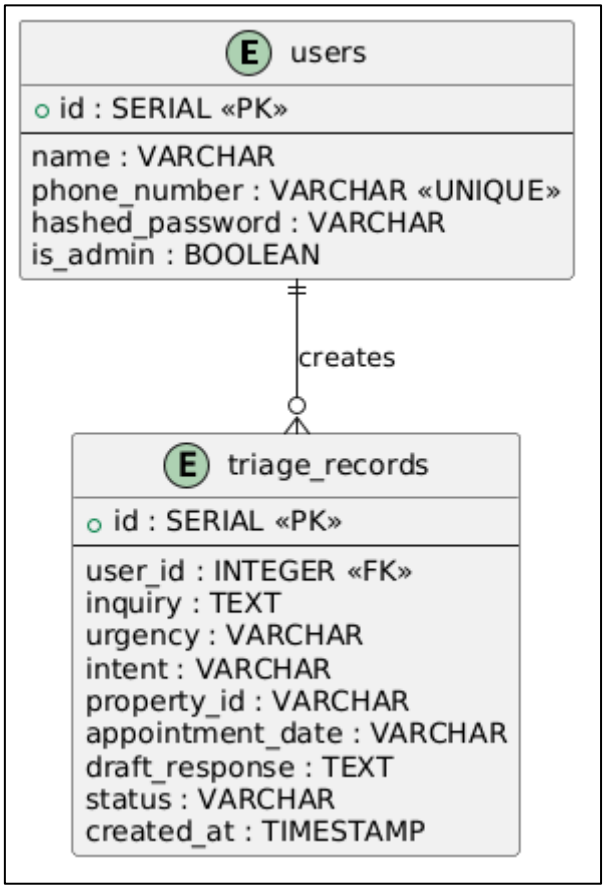
This structure allows administrators to monitor, prioritize, and manage user inquiries efficiently.

iii. Entity Relationship

The relationship between the tables can be summarized as:

User (1) ---------- (Many) TriageRecords

Each inquiry record belongs to exactly one user, while a user may have multiple inquiry records.



E-R Diagram

3.2 Data Access Mechanism

The system accesses the database through **SQLAlchemy ORM**, which provides an abstraction layer between the application logic and the relational database. SQLAlchemy enables the backend to interact with database records using Python objects rather than raw SQL queries. This approach improves code readability, maintainability, and security.

The backend defines ORM models corresponding to the database tables. These models are used to perform operations such as:

- inserting new user records during registration
- retrieving user information during login
- storing inquiry triage results
- fetching inquiry history for users
- retrieving prioritized inquiries for administrators
- updating inquiry status

Database sessions are managed through dependency injection in FastAPI, ensuring that database connections are properly opened and closed during each request lifecycle.

3.3 Data Retention Policies

The system stores user and inquiry data for operational purposes and administrative monitoring.

Currently, the following data retention policies apply:

- User account data is retained for the duration of the account's existence.
- Inquiry records are stored indefinitely to maintain a historical record of customer interactions.
- Administrative updates such as inquiry status changes are preserved for tracking resolution workflows.

In future deployments, retention policies may include automated archival or deletion of outdated inquiry records to optimize storage and maintain compliance with data privacy regulations.

3.4 Data Migration

Database schema changes are managed using **Alembic**, a database migration tool commonly used with SQLAlchemy.

Alembic enables controlled schema evolution by maintaining versioned migration scripts that define incremental changes to the database structure.
Key migration operations include:

- creating new tables
- modifying column structures
- adding constraints or indexes
- updating relationships between entities

Each migration is tracked through a version table that records the current database schema state. This approach ensures that database changes can be consistently applied across development, testing, and production environments.

---

4. Interfaces

The Real Estate AI Triage System provides multiple interfaces that enable interaction between users, administrators, and backend services. These interfaces ensure seamless communication between the frontend application, backend APIs, and the AI processing pipeline.
The system primarily exposes three categories of interfaces:

- User Interface
- Administrative Interface
- API Interface

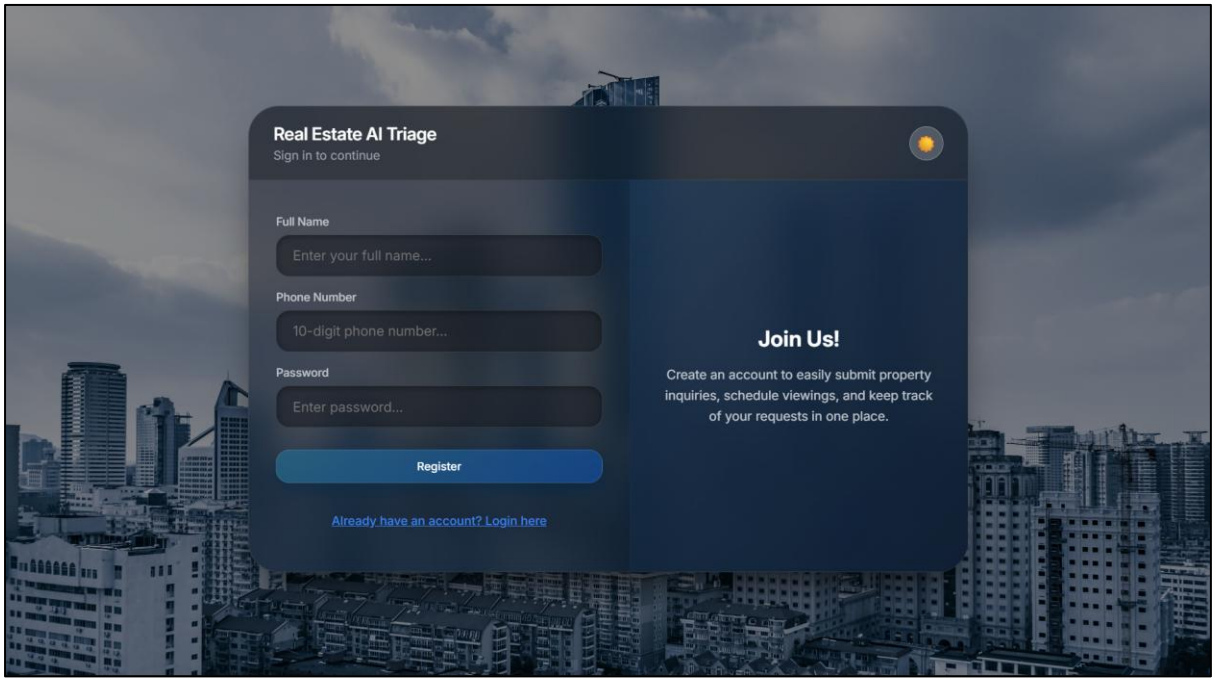
4.1 User Interface

The user interface is implemented using **React with Vite** and serves as the primary interaction layer for end users. It allows registered users to access the system, submit real estate inquiries, and track the status of their previous requests.

The user interface consists of the following main components:

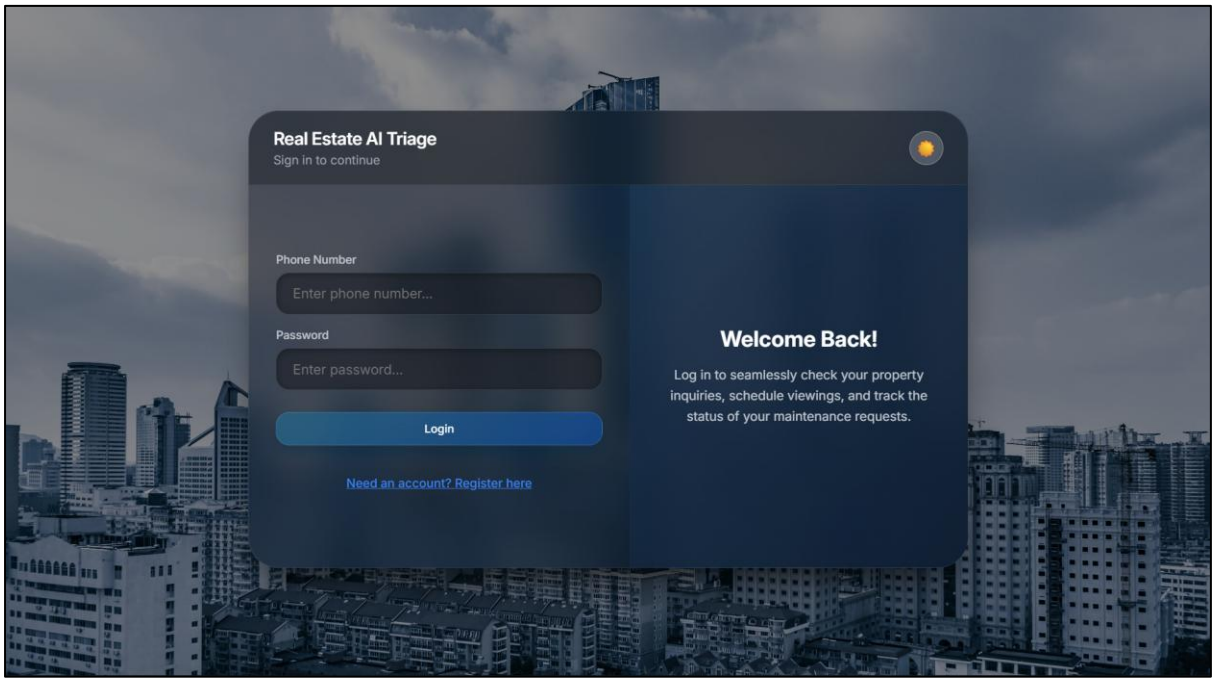
i. User Registration Page

The registration page allows new users to create an account by providing their name, phone number, and password. The information is validated and sent to the backend registration API for account creation.



ii. User Login Page

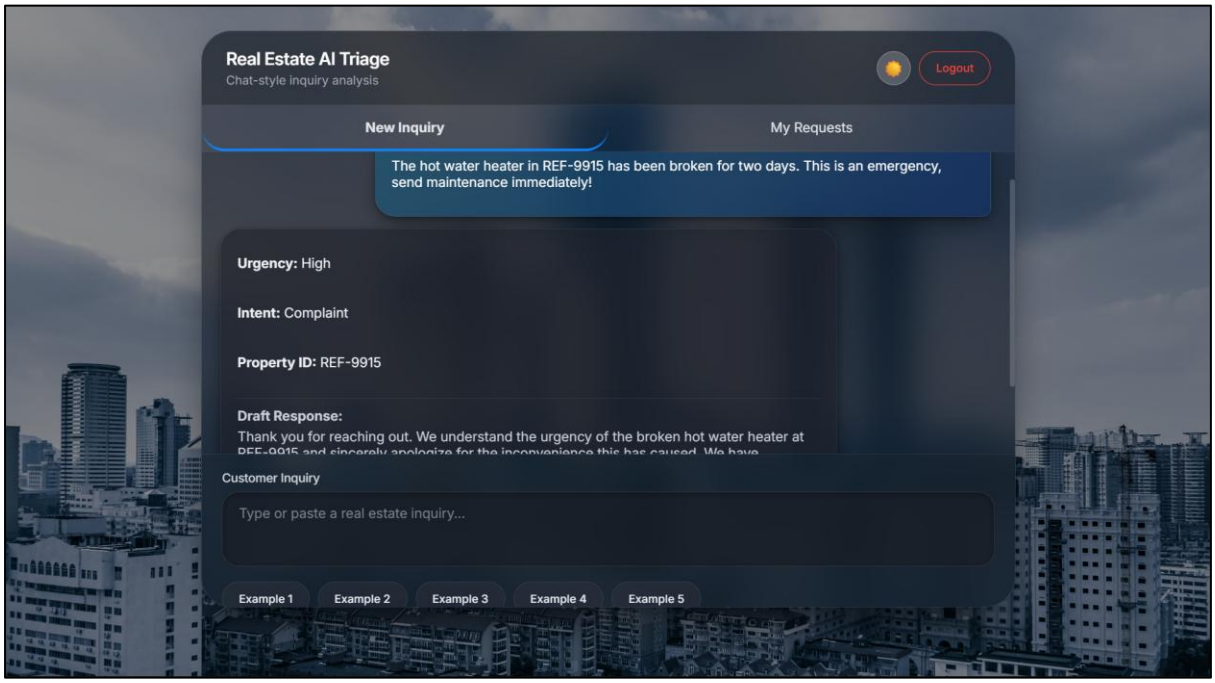
The login interface enables registered users to authenticate themselves using their phone number and password. After successful authentication, the system returns a JWT token which is used for subsequent API requests.



iii. Inquiry Submission Interface

After logging in, users can submit real estate inquiries through the inquiry dashboard. The inquiry message is sent to the backend triage service, which processes it using the AI pipeline.

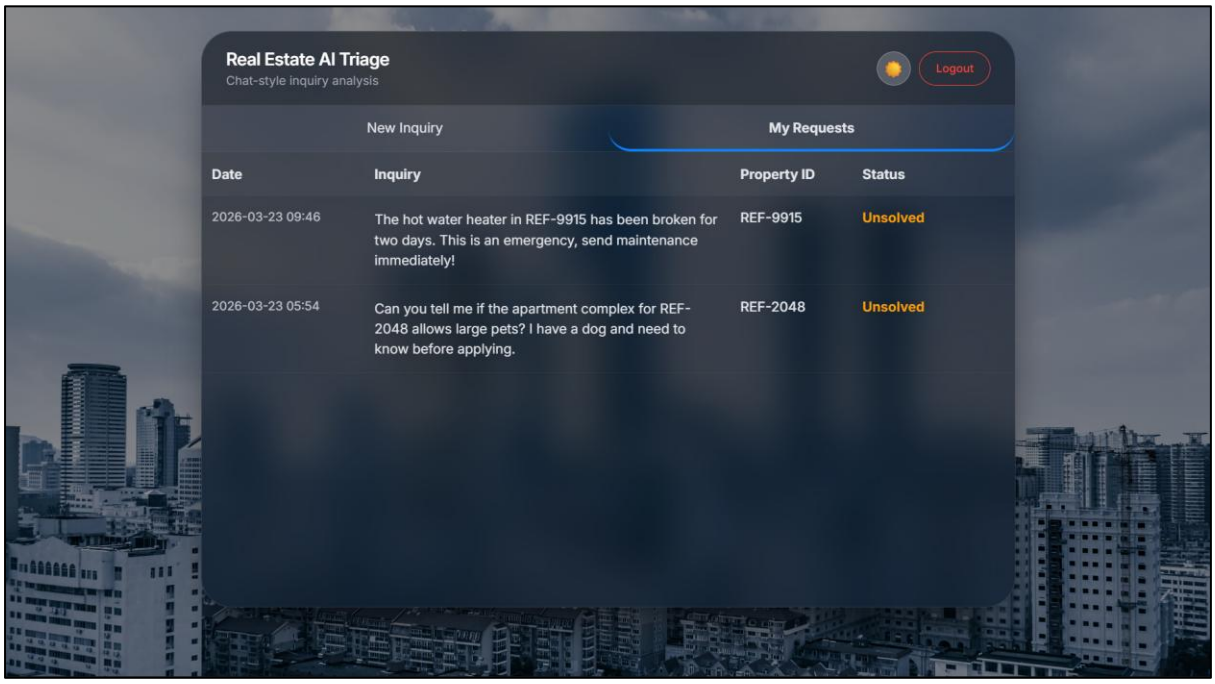
The interface displays the AI-generated analysis including urgency, intent classification, extracted property reference, and draft response.



iv. User Request History Interface

Users can view their previously submitted inquiries through the **My Requests** section. The interface displays the inquiry message, property reference, submission time, and current

status of the request.
This feature allows users to track whether their request is unresolved, under action, or solved.

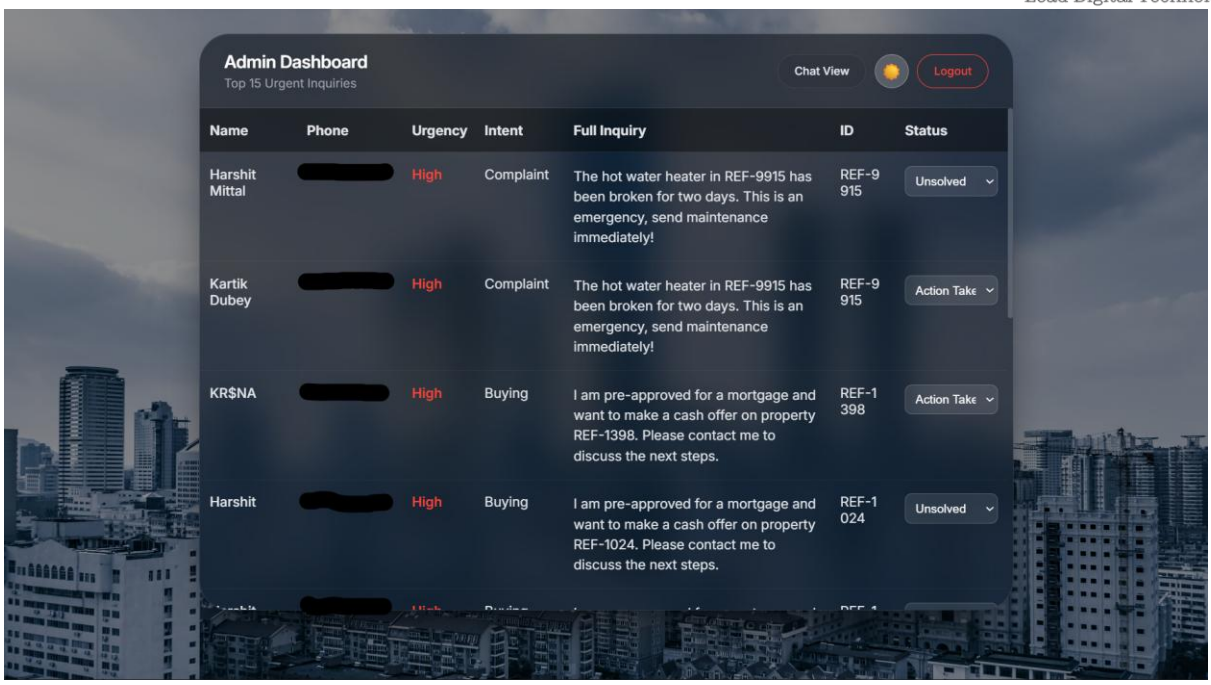


4.2 Administrative Interface

The system includes a dedicated administrative interface that allows authorized administrators to monitor and manage customer inquiries.
Administrators can access the dashboard after authentication using an admin account.
The admin dashboard provides the following capabilities:

- View the most recent inquiries submitted by users
- Display user name and contact information
- View AI-classified urgency and intent
- Read the original inquiry message
- View extracted property reference
- Update inquiry status

The records are automatically sorted by urgency level so that critical requests receive priority attention.



4.3 API Interface

The backend exposes RESTful APIs implemented using **FastAPI** to facilitate communication between the frontend and backend services.
These APIs handle authentication, inquiry processing, data retrieval, and administrative actions.

The primary API categories include:

Authentication APIs
These APIs manage user registration and login.
Examples include:
POST /register
POST /login

Inquiry Processing APIs
These APIs handle inquiry submission and AI processing.
Example:
POST /triage
This endpoint forwards the inquiry to the AI pipeline and stores the resulting triage record in the database.

User Data APIs
These APIs allow users to retrieve their own inquiry history.
Example:
GET /user/records

Administrative APIs
These APIs provide functionality for administrators to retrieve and manage inquiry records.
Examples include:
GET /admin/records

PATCH /admin/records/{record_id}/status
These APIs allow administrators to view prioritized inquiries and update their resolution status.

---

5. State and Session Management

The Real Estate AI Triage System uses a **stateless authentication mechanism based on JSON Web Tokens (JWT)** to manage user sessions and maintain secure communication between the frontend and backend services.
Unlike traditional session-based systems that store session information on the server, JWT-based authentication allows the system to validate user identity without maintaining server-side session state. This approach improves scalability and simplifies backend architecture.

JWT Authentication Mechanism
When a user successfully logs into the system, the backend authentication service generates a **JWT access token** containing encoded user identity information. The token includes details such as the user's phone number and administrative privileges.
The token is digitally signed by the backend to prevent tampering.
The authentication process follows these steps:

1. The user submits login credentials through the frontend interface.
2. The backend verifies the credentials against the stored user records in the database.
3. If authentication succeeds, the backend generates a JWT token.
4. The token is returned to the frontend as part of the login response.
5. The frontend stores the token and includes it in the **Authorization header** for all protected API requests.

Example header used in authenticated requests:
Authorization: Bearer <JWT_TOKEN>

Session Validation
For every protected API request, the backend performs token validation to ensure that the request originates from an authenticated user.
The validation process includes:

- verifying the digital signature of the token
- checking the token expiration time
- extracting user identity information
- determining whether the user has administrative privileges

If the token is valid, the backend grants access to the requested resource. If the token is invalid or expired, the request is rejected and the user must log in again.

Session Expiration
The JWT token includes an expiration timestamp that limits the duration of a user session. This ensures that authentication tokens cannot be used indefinitely.
After the token expires, users are required to log in again to obtain a new token. This mechanism improves system security by reducing the risk of unauthorized access.

Role-Based Access Control
The system uses a simple **role-based access control mechanism** to distinguish between regular users and administrators.

The JWT token contains a flag indicating whether the user has administrative privileges.
Backend services verify this flag when accessing administrative endpoints.
For example:

- regular users can submit inquiries and view their own records
- administrators can retrieve all inquiry records and update inquiry status

This mechanism ensures that administrative operations are restricted to authorized users only.

---

6. Caching

Caching is a performance optimization technique used to temporarily store frequently accessed data so that it can be retrieved quickly without repeatedly querying the database or performing expensive computations.

In the current implementation of the Real Estate AI Triage System, no dedicated caching mechanism is implemented. All requests are processed directly by the backend services, and data is retrieved from the PostgreSQL database whenever needed.
This design decision was made to keep the system architecture simple during the initial development phase. Since the application currently handles a moderate number of requests, direct database access does not introduce significant performance overhead.
However, as the system scales and the volume of user inquiries increases, caching mechanisms may be introduced to improve system performance.

Possible caching strategies that can be integrated in future versions include:

- **Query Result Caching**
  Frequently accessed data such as recent inquiry records or administrative dashboards can be cached to reduce database load.
- **Session Token Validation Caching**
  Short-term caching of validated authentication tokens can reduce repeated token verification overhead.
- **AI Response Caching**
  Similar inquiries submitted by different users could potentially reuse previously generated AI responses, reducing repeated calls to the AI processing pipeline.

Technologies such as **Redis** or **in-memory caching layers** can be integrated with the backend to implement these caching strategies in future deployments.

Currently, the system prioritizes **data consistency and simplicity over caching optimization**, ensuring that all displayed information reflects the most up-to-date database state.

---

7. Non-Functional Requirements

Non-functional requirements define the operational qualities and constraints of the Real Estate AI Triage System. These requirements ensure that the system remains secure, efficient, and reliable when handling real-world user interactions and administrative operations.
The system design considers both security and performance aspects to maintain a stable and trustworthy platform.

7.1 Security Aspects

Security is a critical component of the Real Estate AI Triage System since the platform stores user credentials, inquiry data, and administrative records.
The system implements several mechanisms to protect user data and prevent unauthorized access.

User Authentication
The system uses **JWT-based authentication** to verify the identity of users and administrators. When a user successfully logs in, the backend generates a signed JSON Web Token that is returned to the frontend. This token must be included in the Authorization header for all protected API requests.
This mechanism ensures that only authenticated users can access system resources.

Password Protection
User passwords are never stored in plain text. Instead, passwords are securely hashed before being stored in the database. During login, the system verifies the entered password against the stored hash.
This approach prevents exposure of sensitive credentials even if the database is compromised.

Authorization Control
The system implements role-based access control to differentiate between normal users and administrators.

- Regular users can submit inquiries and view their own request history.
- Administrators can access all inquiry records and update inquiry status.

Administrative privileges are verified using the is_admin flag stored in the user record.

Secure API Access
All sensitive operations such as inquiry submission, record retrieval, and status updates require authentication tokens. The backend verifies the validity of the token before processing any protected request.
Unauthorized access attempts are rejected with appropriate HTTP error responses.

7.2 Performance Aspects

Performance considerations ensure that the system remains responsive and capable of handling user requests efficiently.

Efficient Request Processing
The backend is implemented using **FastAPI**, which provides high-performance asynchronous

request handling. This allows the system to process multiple API requests concurrently without significant delays.

Optimized Database Queries
The system uses PostgreSQL as the primary data storage layer. Queries are structured to efficiently retrieve relevant records, such as retrieving inquiries sorted by urgency for the admin dashboard.
Database indexes and foreign key relationships help improve query performance and maintain data integrity.

AI Processing Efficiency
The AI triage pipeline uses **Gemini Flash**, which is optimized for fast response generation. This ensures that inquiry classification and response generation occur with minimal latency. Additionally, the multi-agent CrewAI pipeline organizes processing tasks efficiently by distributing responsibilities among specialized agents.

Scalability Considerations
The modular architecture allows the system to scale if the number of users and inquiries increases.
Possible scaling strategies include:

- deploying multiple backend instances
- introducing caching mechanisms
- optimizing AI inference pipelines

These improvements can help maintain system responsiveness under higher workloads.

---

8. References

The following resources and technologies were referenced during the design and development of the Real Estate AI Triage System:

1. **FastAPI Documentation** – Official documentation for building high-performance REST APIs using FastAPI.
   https://fastapi.tiangolo.com/
2. **React Documentation** – Official documentation for building user interfaces using the React JavaScript library.
   https://react.dev/
3. **CrewAI Documentation** – Documentation for implementing multi-agent AI workflows using CrewAI.
   https://docs.crewai.com/
4. **Google Gemini API Documentation** – Documentation for integrating Gemini language models for natural language processing tasks.
   https://ai.google.dev/
5. **PostgreSQL Documentation** – Official documentation for PostgreSQL relational database management system.
   https://www.postgresql.org/docs/
6. **JWT (JSON Web Token) Specification** – Documentation describing the JWT standard used for authentication.
   https://jwt.io/